

Thread-level Parallelism

Sistemas de Computadores

Departamento de Engenharia Informática

Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Overview

- **Parallel computing hardware**
- **Cache coherency and sequential consistency**
- **Exploiting thread-level parallelism**

Introduction

- **In the previous lecture, we focused on threads and correctness**
 - Avoiding race conditions
 - Writing thread-safe functions
 - Ensuring correct behavior under concurrency
- **But correctness alone may not be enough**
 - A program can be correct, **yet still perform poorly**
- **New challenge!**
 - Understanding how hardware affects concurrent programs

Introduction

■ Performance depends on:

- Caches
- Memory access patterns
- Processor and compiler behavior

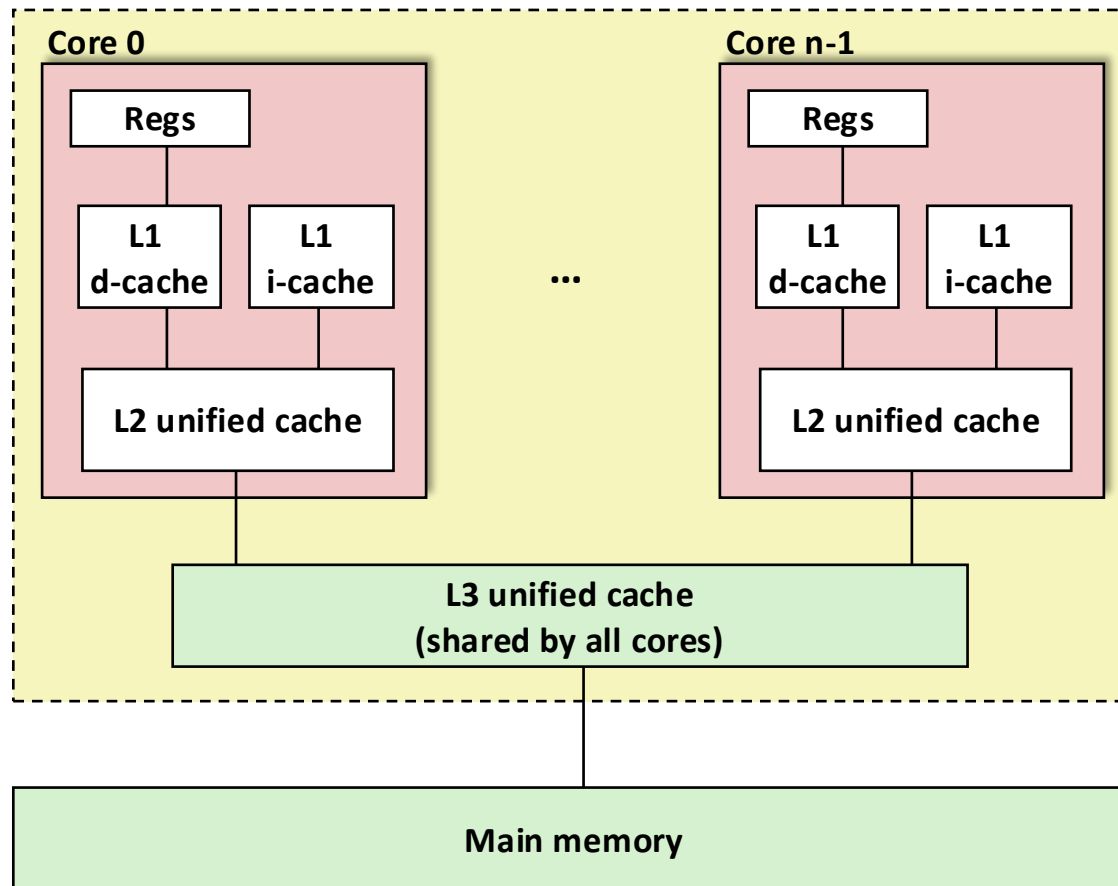
■ This lecture:

- How modern processors execute concurrent code
- Why naïve parallel programs can be slow or unpredictable
- How to design efficient parallel programs

Symmetric shared-memory systems

- **Modern high-performance processors have increasing memory demands**
 - More cores means more simultaneous memory accesses
- **But traditional shared buses become a bottleneck and do not scale well**
- **Multiple processors/cores share a common address space**
 - Each core typically has private caches (local, fast access) to reduce memory access latency
 - But access a shared main memory (larger, slower)

Typical multicore processor



Symmetric shared-memory systems

■ Private data

- Used by a single core
- Stored and accessed locally

■ Shared data

- Accessible by multiple cores
- Enables communication via reads and writes

■ Key challenge

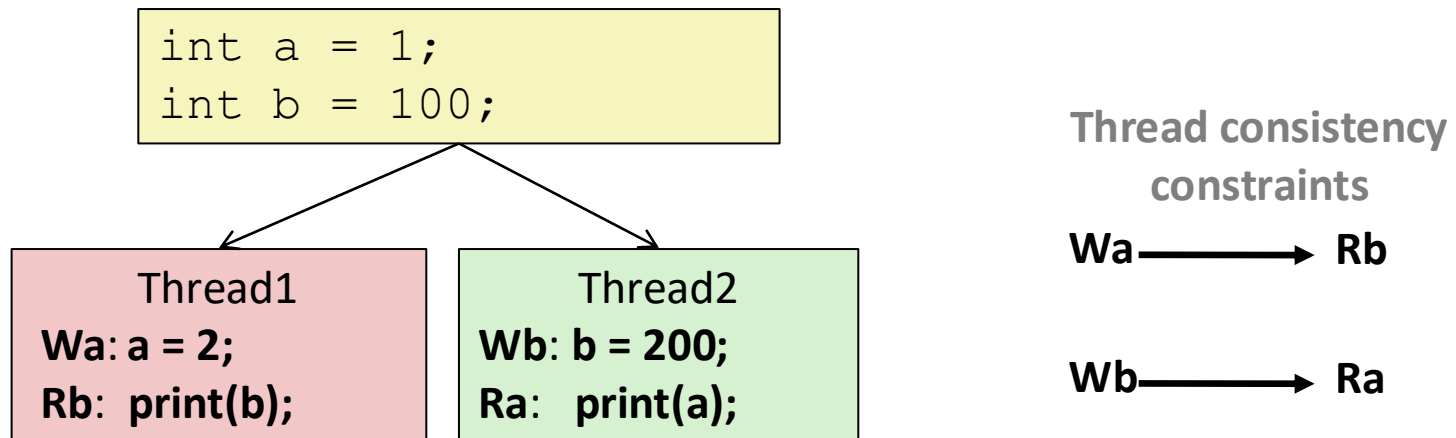
- Ensuring all cores see a **consistent view of shared data**

Overview

- Parallel computing hardware
- **Cache coherency and memory consistency**
- Exploiting thread-level parallelism

The challenge

- Consider two threads running on different cores accessing shared variables
 - What are the possible values printed?



- Depends primarily on the **memory consistency model**, assuming **cache coherence** is ensured

Cache coherency

■ Consistent view of shared data

- All processors eventually observe the **same value** for a memory location
- Writes to the same location are seen in a **coherent (serialized)** order

■ Coherence protocols ensure updates become visible to other cores

- Typically, via invalidation (other cached copies are marked invalid), or update mechanisms

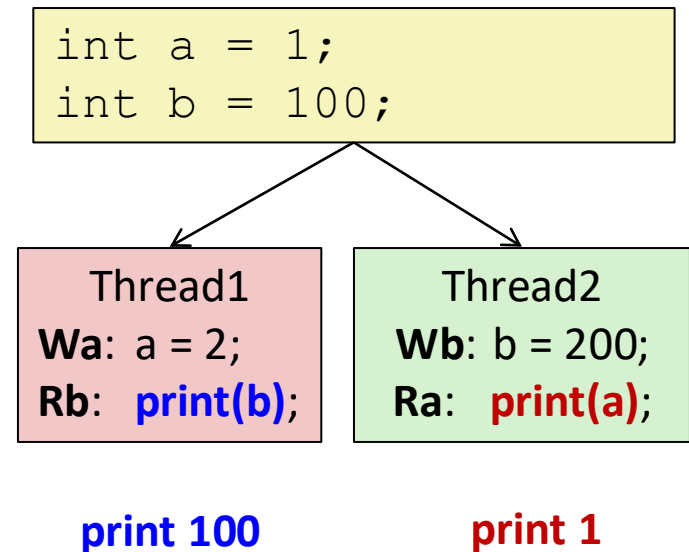
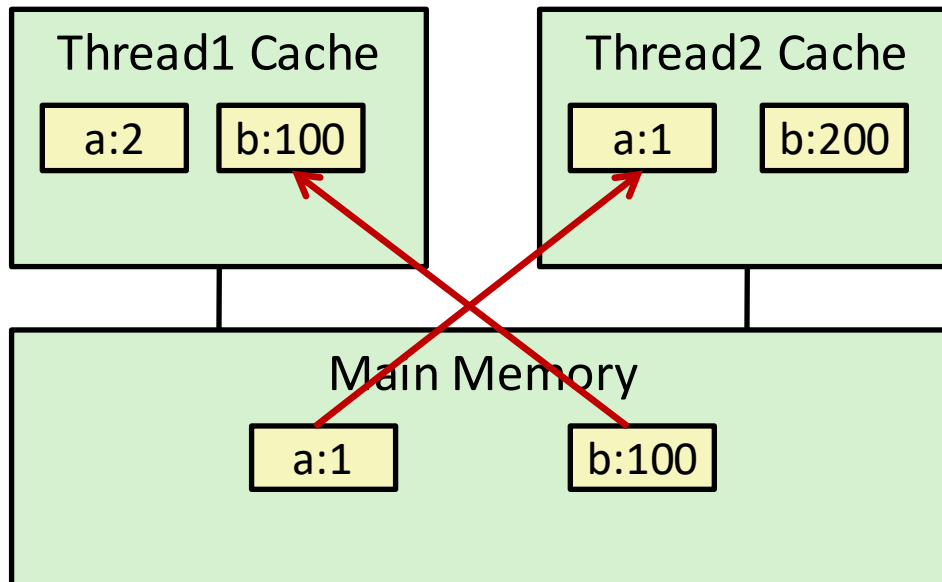
■ Without cache coherence:

- Different cores may see different (stale) values
- Leads to incorrect and unpredictable behavior

Non-coherent cache scenario

■ Write-back caches, without coordination between them

- Updates remain in the cache and are written to memory later
- Other cores may read stale data

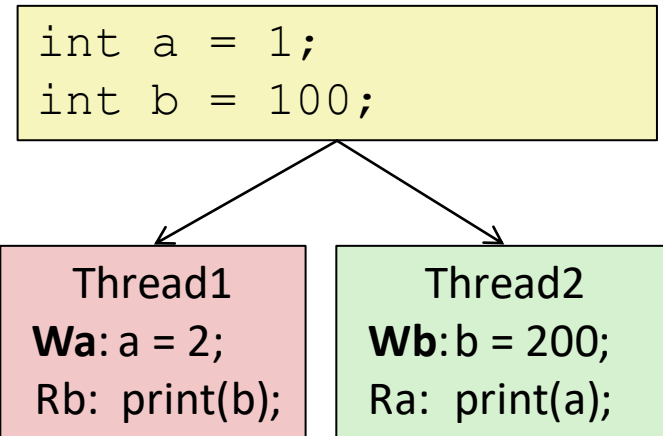
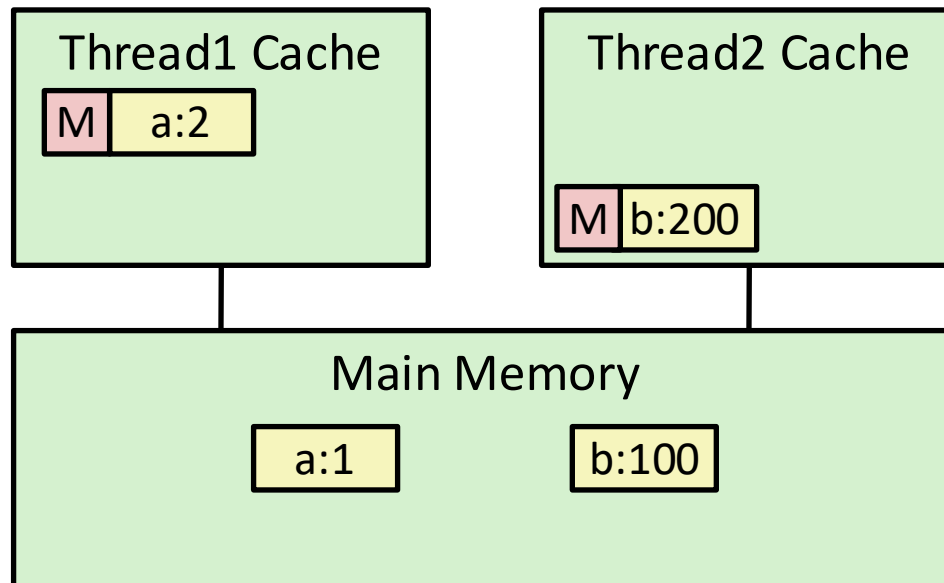


At later points, `a : 2` and `b : 200` are written back to main memory when the cache line is replaced

Snoopy caches

■ Tag each cache block with state

Invalid	Cannot use value
Shared	Readable copy
Modified	Writeable copy

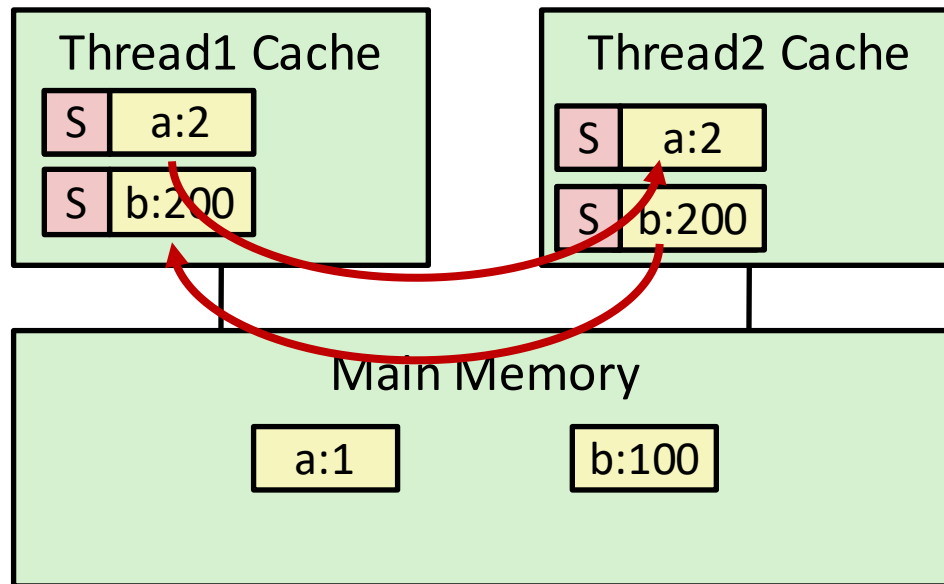


Have cache watch (or **snoop** upon) DMA transfers, and then “do the right thing”

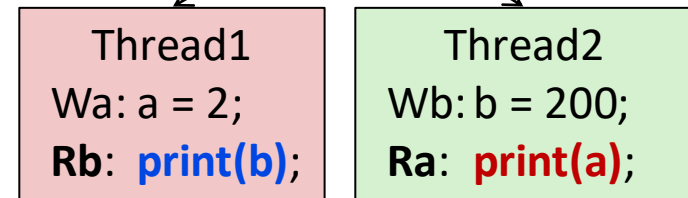
Snoopy caches

■ Tag each cache block with state

Invalid	Cannot use value
Shared	Readable copy
Modified	Writeable copy



```
int a = 1;
int b = 100;
```



print 200

print 2

- When cache sees request for one of its M-tagged blocks
- Supply value from cache (value in memory may be stale)
- Set tag to S

Is coherence enough?

- Cache coherence ensures that all cores eventually see the **same value for each variable**
- But consider multiple variables and multiple threads accessing them concurrently. Even if each variable is coherent:
 - In what order are operations observed?
 - When does a write become visible to other threads?
- Coherence is about a single location, we also need to **reason about ordering**
 - This leads to memory consistency models

Cache coherency vs memory consistency

■ Cache coherence

- Concerned with a **single memory location**
- Ensures all cores see a consistent value for that location
- Writes to the same variable are seen in a coherent order
- *“What value do I see for this variable?”*

■ Memory consistency

- Concerned with **ordering of operations across multiple locations**
- Defines when and in what order reads/writes become visible
- Affects how threads observe interleavings of operations
- *“In what order do I see operations across variables?”*

Sequential consistency

- **Sequential consistency is the gold standard of memory consistency models**
 - It serves as the baseline definition for how a system should behave to appear “logical” to a programmer
- **Under sequential consistency, all operations:**
 - Appear in a single global order
 - While preserving the program order within each thread
- **Note:**
 - This is a **conceptual model**, not actual hardware behavior!
 - It acts as the benchmark for “correct” behavior

Sequential consistency

■ Why it matters

- Provides a simple and predictable model for reasoning about concurrency

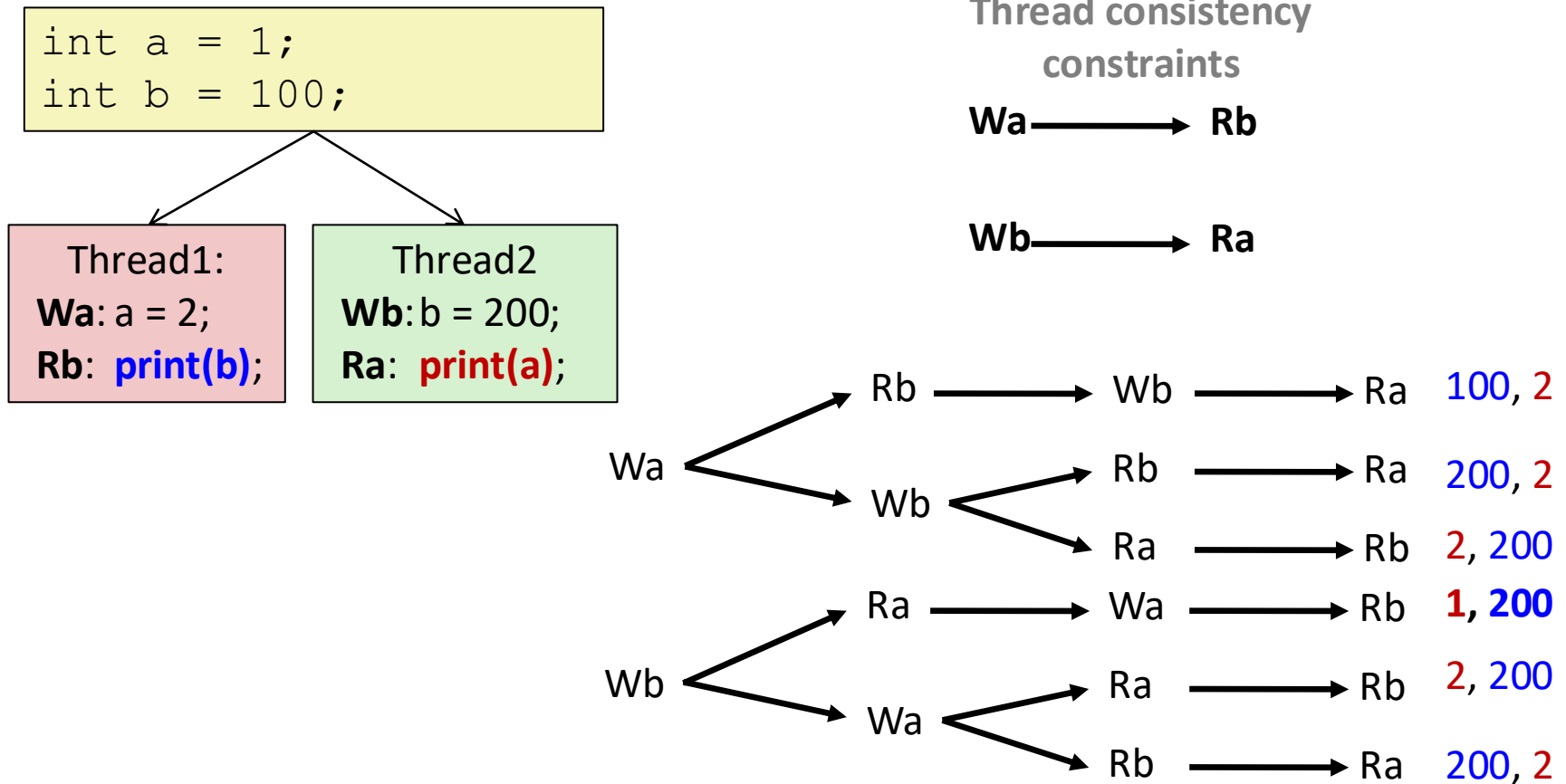
■ Sequential consistency restricts the possible outcomes

- Not all interleavings are allowed

■ This gives us a predictable model

- Even though execution is concurrent, we can still reason about all valid behaviors

Sequential consistency example



■ Impossible outputs: 100, 1 and 1, 100

- These outputs would require violating the program order of at least one thread (reaching **both** Ra and Rb **before either** Wa or Wb)

Relaxed consistency models

- Modern hardware often prioritizes performance over the strict, intuitive ordering of sequential consistency
- This led to the development of **relaxed consistency models**
 - Which allow for certain optimizations at the cost of programmatic simplicity
- Architectures like ARM and RISC-V use relaxed consistency models to boost performance
 - x86 provides a stronger (but still not sequentially consistent) model

Relaxed consistency models

■ Reorder operations

- Let a later read move ahead of an earlier, slower write

■ Buffer writes

- Store data in a local cache or buffer before making it visible to all other processors

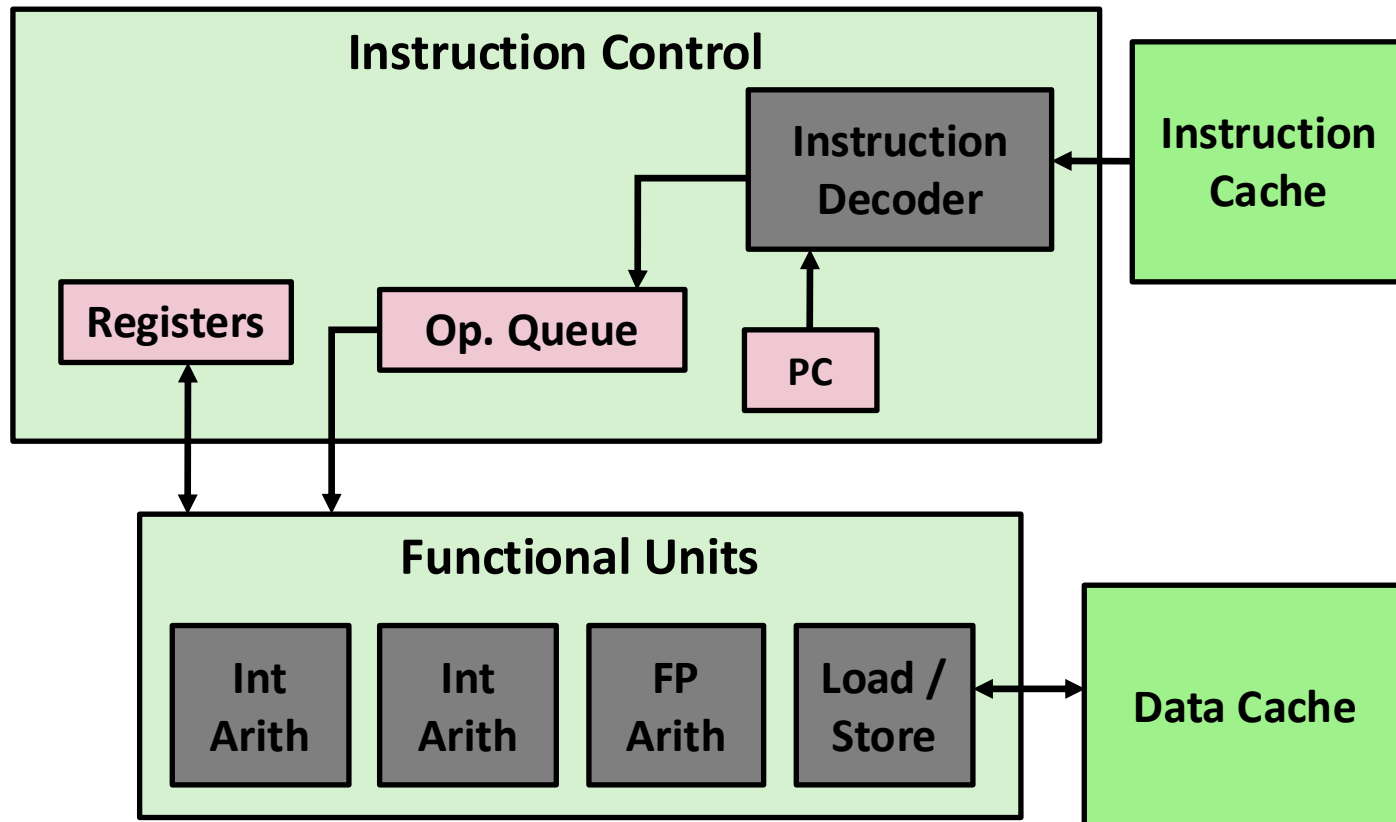
■ Hide latency

- Keep the CPU busy instead of waiting for slow memory interconnects

■ Relaxed models boosts throughput but complicates reasoning in concurrent programs

Out-of-Order processor structure

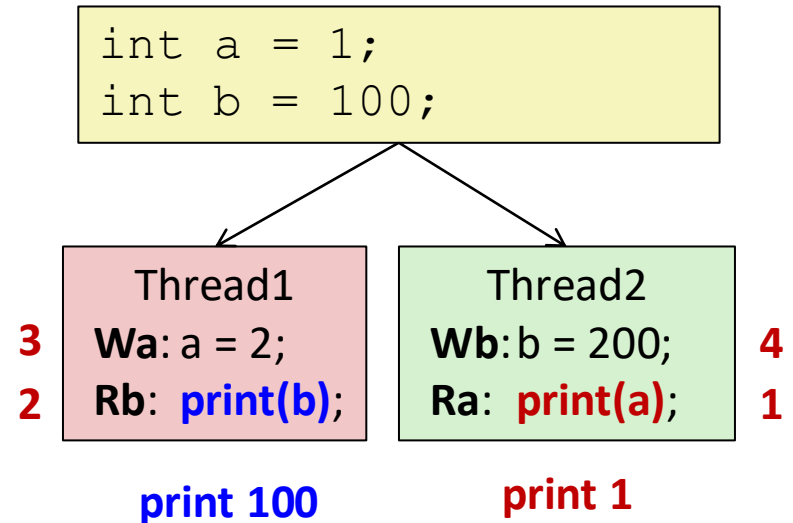
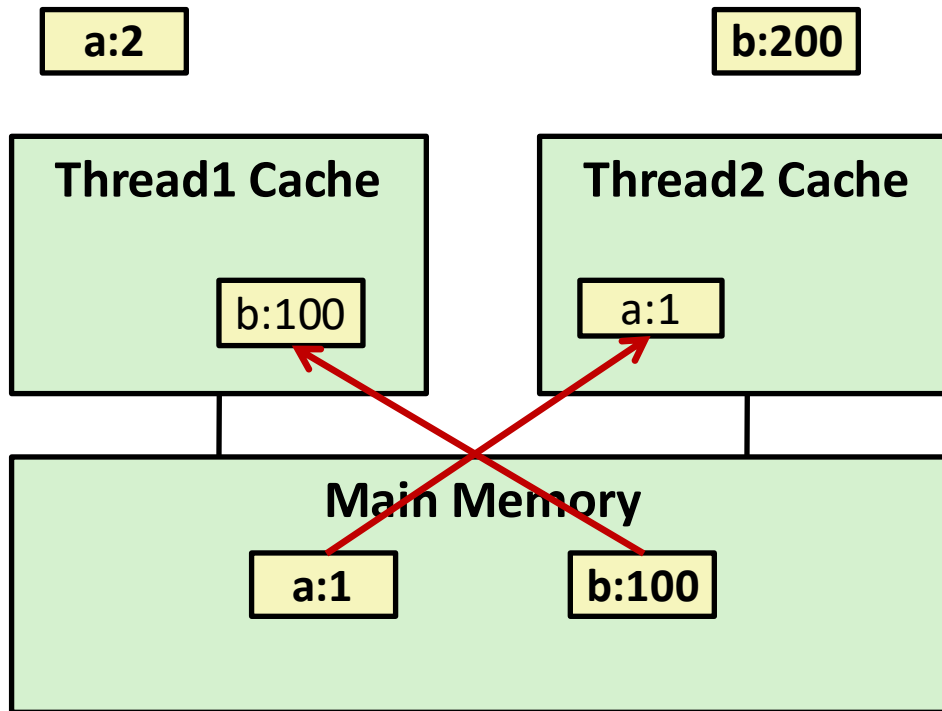
- Instruction control dynamically converts program into stream of operations
 - Operations mapped onto functional units to execute in parallel



Non-sequentially consistent scenario

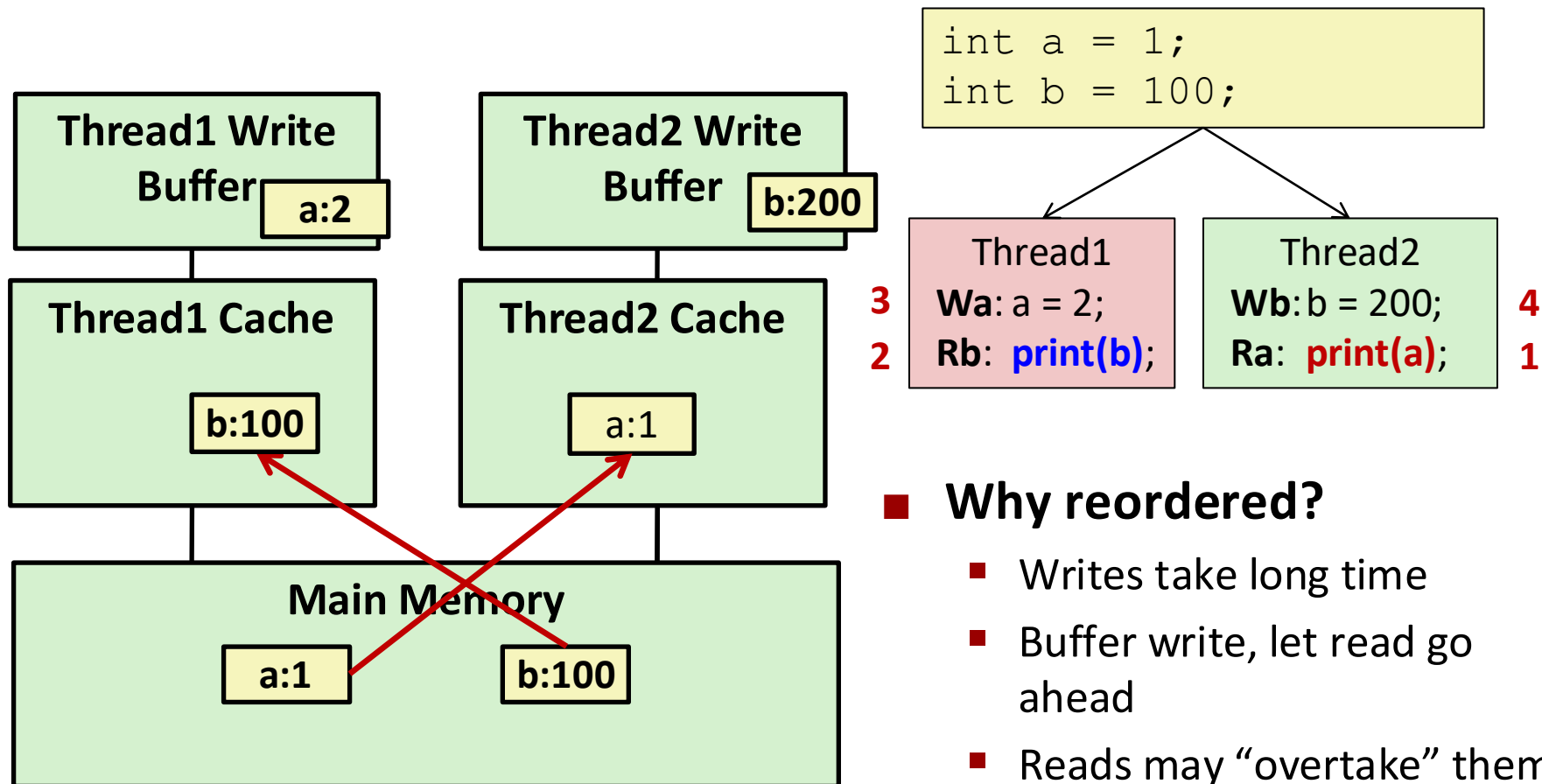
■ Coherent caches

- But thread consistency constraints violated due to **operation reordering**



- Architecture lets reads finish before writes because a single thread accesses different memory locations

Non-sequentially consistent scenario



■ Why reordered?

- Writes take long time
- Buffer write, let read go ahead
- Reads may “overtake” them

Even with coherent caches, reordering can produce results that violate sequential consistency

Why memory barriers are needed

- **Modern processors may reorder operations and delay visibility of writes**
 - This can lead to unexpected results in concurrent programs
 - Because the hardware is “relaxed”, the burden of correctness shifts to the programmer
- **Goals of memory barriers:**
 - Control **ordering and visibility** of memory operations
 - Prevent undesired reordering by the processor or compiler
- **Programmers and compilers must insert the right barriers to avoid subtle bugs**
 - Use instructions like `MFENCE` (x86), `DMB/DSB` (ARM), or `FENCE` (RISC-V) to enforce order at critical points

Types of memory barriers (fences)

■ Acquire barrier

- Prevents later operations from moving before the barrier
- Ensures that all reads/writes after the barrier see up-to-date values

■ Release barrier

- Prevents earlier operations from moving after the barrier
- Ensures that all prior writes become visible before continuing

■ Full barrier

- Prevents all reordering across the barrier
- Ensures all previous operations complete before any subsequent ones begin

The need for synchronization

- **Higher-level synchronization primitives provide the abstraction developers need to write safe code**
- **Primitives like mutexes, semaphores, and condition variables are explicitly implemented using:**
 - Hardware fences to enforce memory ordering
 - Atomic instructions (e.g., CAS, LL/SC) to ensure indivisible updates
- **Takeaway**
 - You need both a coherent hardware substrate and disciplined use of synchronization in software to achieve true sequential consistency

Takeaway

- **Cache coherence is necessary, but not sufficient**

- Ensures agreement on values
- Does not guarantee ordering of operations

- **Threads must observe:**

- Up-to-date values (no stale reads)
- Consistent ordering of operations

- **Sequential consistency provides this model**

- Global ordering of operations
- Program order preserved within each thread

Takeaway

■ Hardware alone is not enough

- Modern processors may reorder operations and delay visibility of writes

■ Programmers must use synchronization primitives

- To control access to shared data
- To enforce ordering and visibility between threads

■ These mechanisms establish happens-before relationships

- The system must behave as if events occurred in the exact order specified by the code
- Making program behavior predictable and correct

Overview

- Parallel computing hardware
- Cache coherency and sequential consistency
- **Exploiting thread-level parallelism**

Exploiting parallel execution

- The goal of threading is to maximize CPU utilization by distributing workloads across N cores
- This is generally achieved through two distinct strategies
 - **Task-level parallelism** (high throughput) – Leveraging multiple CPUs to handle independent flows of execution
 - **Data-level parallelism** (lower latency) – accelerating a single, intensive workload by decomposing it into concurrent sub-tasks
- Even with correct synchronization, performance depends heavily on how data is shared

Case study: Parallel summation (0...N-1)

- **Compute the sum of all integers up to N-1 (e.g., $N = 2^{30}$)**
 - Expected result: $(N-1) * N / 2$
- **Divide the total range into K equal segments**
 - Each of the T threads processes exactly one range of $\lfloor N/K \rfloor$ values
 - Threads calculate their local partial sums simultaneously
 - A single thread (usually `main`) accumulates the leftover values and merges the partial results
- **While the logic is simple, several “hidden” factors often prevent linear speedup**

How to interpret performance results

■ Benchmark machine

- Intel Core i9-9900K
- 3.6 GHz base, up to 5.0 GHz Turbo Boost
- 8 cores x 2 threads per core (Hyper-Threading) = 16 threads

■ Execution time

- Actual performance (what we care about)
- Always compare absolute execution time first

■ Speedup

- Relative improvement over a baseline
- Our baseline is single-threaded, unsynchronized, global accumulation
- Speedup depends on the quality of the baseline

Case study: Parallel summation (0...N-1)

■ Method #1: All threads update a single global variable

- 1A: No synchronization
- 1B: Synchronize with a semaphore
- 1C: Synchronize with a `pthread_mutex`

```
/* single accumulator */  
typedef unsigned long data_t;  
volatile data_t global_sum;  
  
/* mutex and semaphore for global_sum */  
sem_t semaphore;  
pthread_mutex_t mutex;  
  
/* number of elements summed by each thread */  
size_t nelems_per_thread;  
  
/* keep track of thread IDs */  
pthread_t tid[MAXTHREADS];  
  
/* identify each thread */  
int myid[MAXTHREADS];
```

Accumulating in single global variable

- The main thread creates multiple worker threads that all contribute to a single shared accumulator

```
nelems_per_thread = nelems / nthreads;
global_sum = 0;

/* Create threads */
for(i = 0; i < nthreads; i++){
    /* each thread handles a different segment of the array */
    myid[i] = i;
    pthread_create(&tid[i], NULL, thread_func, &myid[i]);
}

/* wait for threads to finish */
for (i = 0; i < nthreads; i++)
    pthread_join(tid[i], NULL);

result = global_sum;

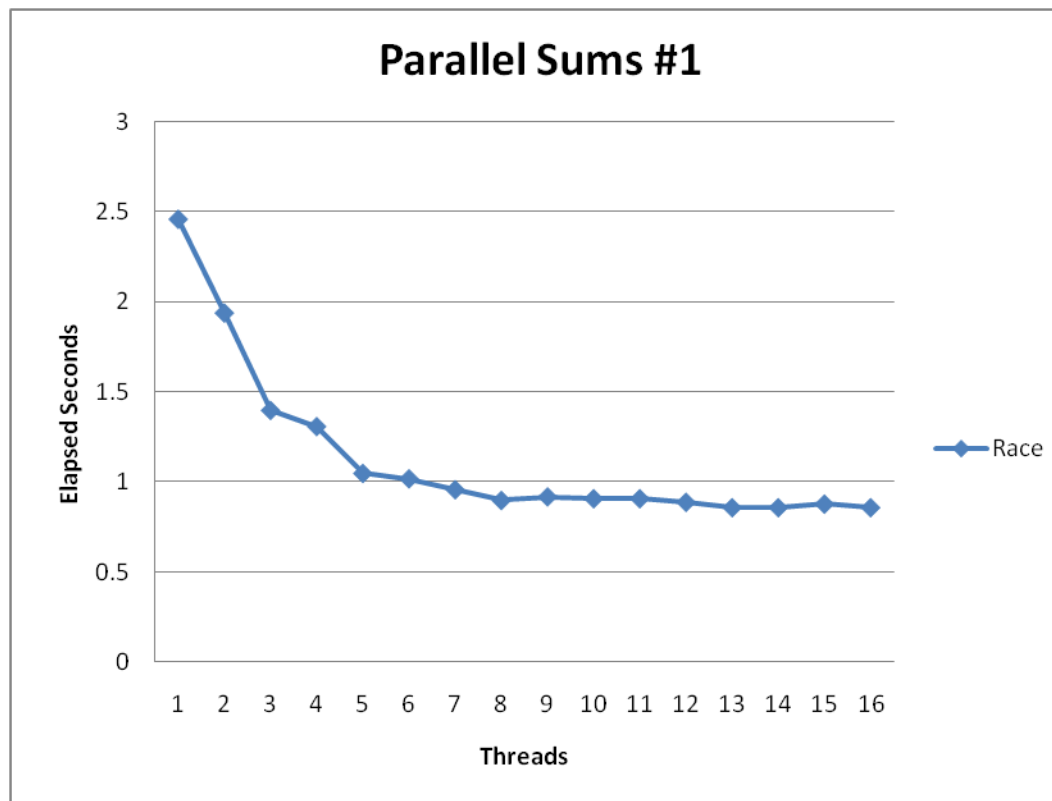
/* Add leftover elements */
for(e = nthreads * nelems_per_thread; e < nelems; e++)
    result += e;
```

Thread function: No synchronization

- Each thread directly updates a shared global variable without any synchronization
 - Leading to race conditions and incorrect results, as expected

```
void *sum_race(void *arg){  
    int myid = *((int *)arg);  
    size_t start = myid * nelems_per_thread;  
    size_t end = start + nelems_per_thread;  
    size_t i;  
  
    for(i = start; i < end; i++)  
        global_sum += i;  
  
    return NULL;  
}
```

Unsynchronized performance



■ Execution time:

- Fast but incorrect
- Gets **wrong answer** when > 1 thread!

■ Speedup:

- Up to 2.8x with 16 threads
- Baseline uses 1 thread

The failure of `volatile`

- `volatile` prevents compiler optimizations, but **it does not guarantee** atomicity, ordering, or visibility across threads
 - Leads to race conditions and incorrect results
 - Fix: enforce synchronization on shared data access
- The “parallelism paradox”
 - A shared global counter becomes a performance bottleneck
 - Adding more threads actually increases the amount of work the hardware has to do to keep memory consistent

Thread function: semaphore/mutex

- Each update to the shared variable is protected by a lock
 - Ensuring correctness but introducing high contention

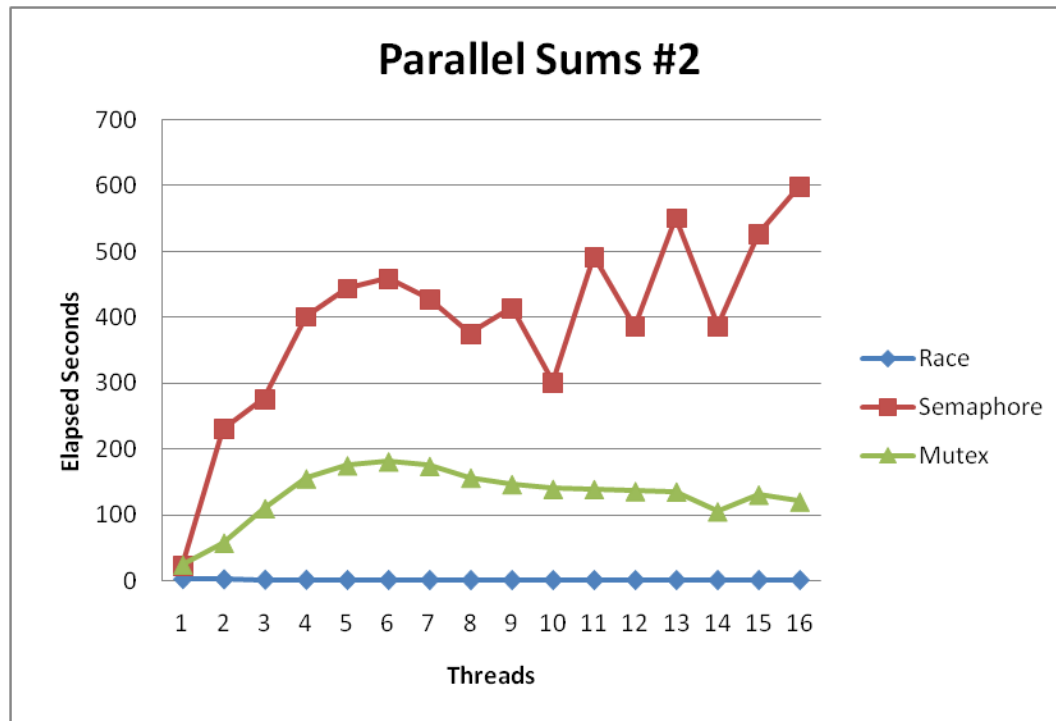
Semaphore

```
void *sum_sem(void *arg) {  
    int myid = *((int *)arg);  
    size_t start = myid * nelems_per_thread;  
    size_t end = start + nelems_per_thread;  
    size_t i;  
  
    for(i = start; i < end; i++){  
        sem_wait(semaphore);  
        global_sum += i;  
        sem_post(semaphore);  
    }  
    return NULL;  
}
```

Mutex

```
pthread_mutex_lock(&mutex);  
global_sum += i;  
pthread_mutex_unlock(&mutex);
```

Semaphore/mutex performance



■ Execution time:

- Extremely slow (2.5 s to ~10 min)
- High contention and serialization due to frequent locking on a shared variable

■ Speedup:

- $< 1x$ (worse than sequential)

Case study: Parallel summation (0...N-1)

- **Method #2: Each thread accumulates into a separate variable**
 - 2A: Accumulate in contiguous array elements
 - 2B: Accumulate in spaced-apart array elements
 - 2C: Accumulate in CPU registers
- **The main thread initializes per-thread accumulators and later combines their partial results into the final sum**

```
/* Partial sum computed by each thread */  
data_t psum[MAXTHREADS * MAXSPACING];  
  
/* Spacing between accumulators */  
size_t spacing = 1;
```


Thread function: Contiguous memory accumulation

- Each thread accumulates results in its own memory location
 - Reducing contention compared to a shared global variable

```
void *sum_global(void *arg){
    int myid = *((int *)arg);
    size_t start = myid * nelems_per_thread;
    size_t end = start + nelems_per_thread;
    size_t i;

    size_t index = myid * spacing;
    psum[index] = 0;
    for (i = start; i < end; i++) {
        psum[index] += i;
    }
    return NULL;
}
```

Thread function: Separate accumulation

```
nelems_per_thread = nelems / nthreads;

/* Create threads */
for(i = 0; i < nthreads; i++){
    myid[i] = i;
    psum[i*spacing] = 0;
    pthread_create(&tid[i], NULL, thread_func, &myid[i]);
}

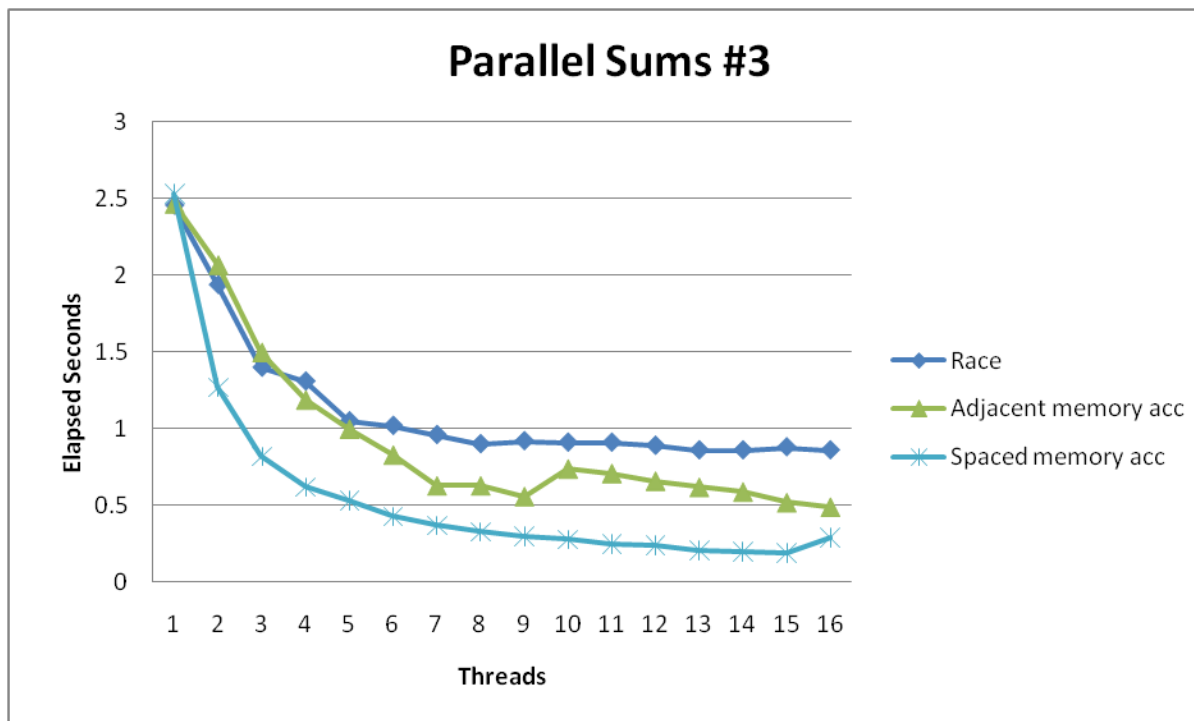
/* Wait for them to finish */
for(i = 0; i < nthreads; i++)
    pthread_join(tid[i], NULL);

result = 0;

/* Add up the partial sums computed by each thread */
for(i = 0; i < nthreads; i++)
    result += psum[i*spacing];

/* Add leftover elements */
for(e = nthreads * nelems_per_thread; e < nelems; e++)
    result += e;
```

Memory accumulation performance



■ Execution time:

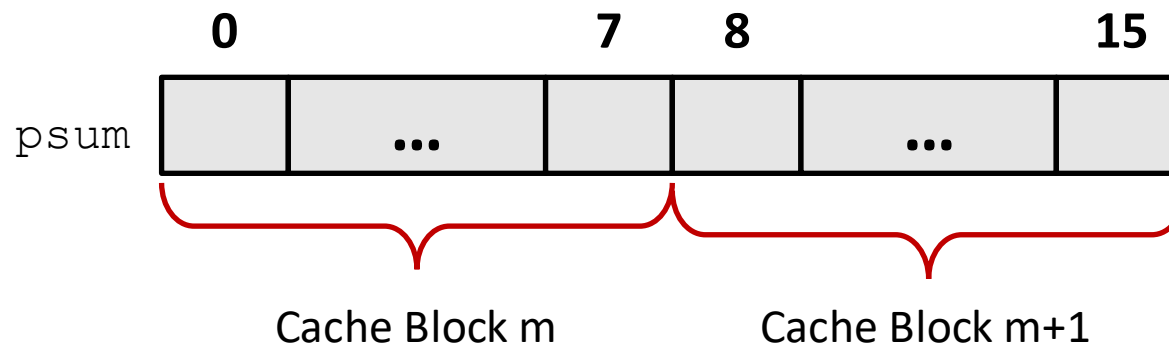
- Adjacent: moderate
- Spaced-apart: much faster

■ Speedup:

- Adjacent: 5x
- Spaced-apart: 13.3x

False sharing

■ Why does spacing the accumulators apart matter?



■ A cache block contains more than one word

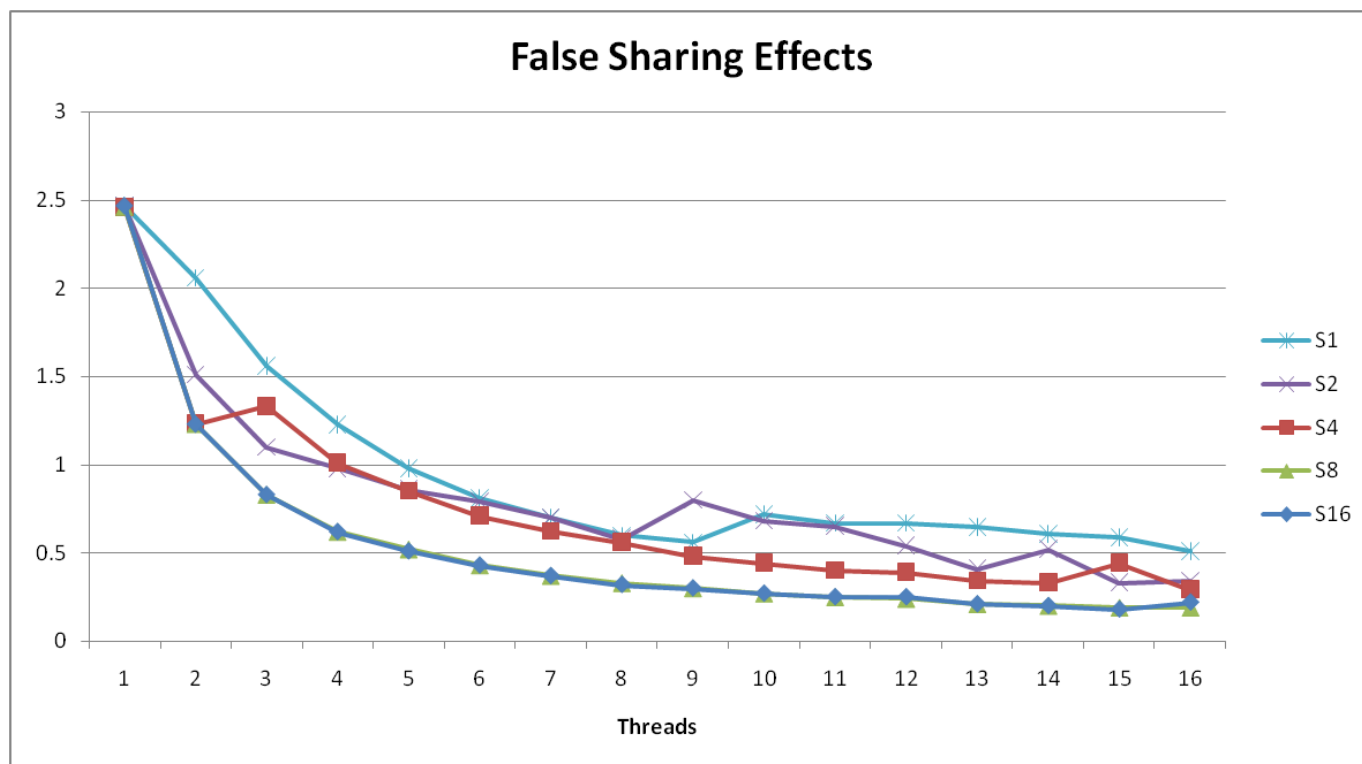
- Cache-coherence is done at the block-level and not word-level

■ To update `psum[i]`, thread `i` must have exclusive access

- Threads sharing common cache block will keep fighting each other for access to block

False sharing

- **False sharing occurs when independent variables share the same cache line, causing unnecessary coherence traffic**
 - Experiment confirms that cache block size is 64 bytes (8 x 8-byte values)
 - No benefit increasing spacing beyond 8



False sharing

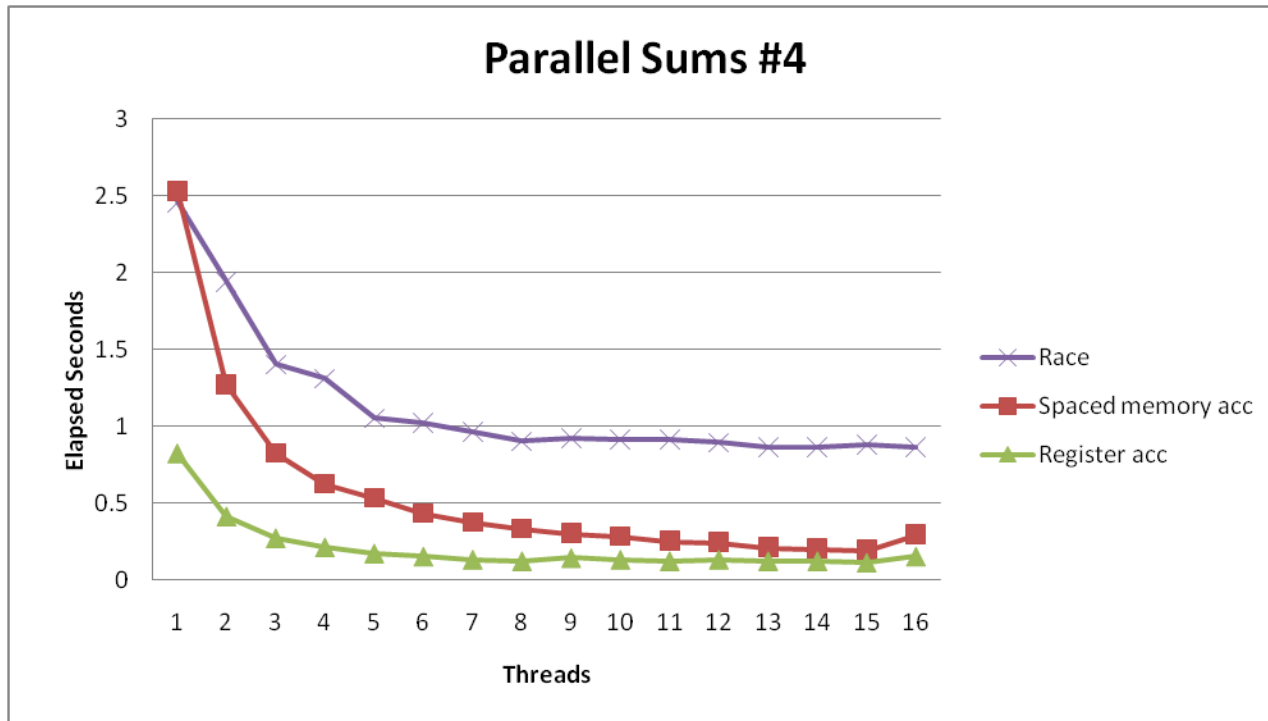
- **Eliminating false sharing unlocks hardware performance**
 - Threads update different variables, but variables are in the same cache line
 - Hardware treats it as shared data, causing unnecessary contention
- **The spaced-apart approach has a significant speedup (13.3x)**
 - Removes a huge inefficiency (false sharing + memory contention)
- **Looks very good relative to our baseline**
 - But our baseline **is memory-bound and suboptimal**

Thread function: Register accumulation

- Each thread performs the entire accumulation locally in a register and writes the result once in memory
 - Minimizing memory access and contention

```
void *sum_local(void *arg) {  
    int myid = *((int *)arg);  
    size_t start = myid * nelems_per_thread;  
    size_t end = start + nelems_per_thread;  
    size_t i;  
    size_t index = myid*spacing;  
  
    data_t sum = 0;  
    for(i = start; i < end; i++){  
        sum += i;  
    }  
    psum[index] = sum;  
    return NULL;  
}
```

Register accumulation performance



■ Execution time:

- Fastest overall

■ Speedup:

- Up to 7.5x
- $\approx 2x$ faster than best memory-based approach
- Speedup alone can be misleading

Lessons learned

- **Always compare your parallel speedup to the best single-thread implementation you can write**
 - The spaced-apart memory version looks faster relative to the baseline, but the register version is actually faster in absolute terms
- **Account for sharing costs**
 - True sharing: multiple threads reading/writing the same data
 - False sharing: independent variables packed on one cache line still incur coherence traffic
- **The fastest parallel code:**
 - Avoids sharing during computation (computation stays local)
 - Shares only final results (memory access is minimized)

Understanding performance results

- **Different designs stress different resources:**
 - Shared variable → contention
 - Memory → bandwidth / cache effects
 - Registers → CPU limits
- **Speedup depends on the bottleneck removed**
 - Large speedup is different from the fastest solution
- **It is not just how to parallelize, but how to reason about performance rigorously**
 - Always evaluate correctness, execution time, bottleneck

Summary

- **Parallel programs must be designed for both correctness and performance – one does not guarantee the other**
 - Correctness is mandatory
 - Performance depends on how well hardware is used

- **Correctness requires:**
 - Synchronization
 - Proper memory ordering

- **Performance requires:**
 - Minimizing sharing
 - Avoiding contention
 - Keeping computation local